

Deployment Guide

Run the trained invalid-traffic network as a shadow/advisory scoring service

The deployment half of the pipeline: take the JNeopallium network the trainer produced, load it into the runtime scorer, expose it as an HTTP scoring service beside your ad stack, and run it as a continuous shadow-to-advisory invalid-traffic scorer that never blocks or withholds on its own.

Document	Production Deployment Guide
Product	Jneopallium Ad-Fraud Guardian (advertising-fraud module)
Consumes	The deployable network from the Training Guide
Safety mode	ADVISORY / SHADOW (recommend-only)
Interfaces	HTTP scoring service + embeddable runtime scorer
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	23 June 2026
License	BSD 3-Clause

Contents

1. What you will achieve
2. Prerequisites — the trained bundle and runtime
3. The runtime scorer and the model bundle
4. The deployable package (what training produced)
5. Option A — the HTTP scoring service
6. Option B — the embedded worker network
7. The advisory decision contract
8. Configure event sources and streaming input
9. Promote: shadow → advisory → bounded enforcement
10. Monitoring in production
11. Safety, privacy and rollback
12. Troubleshooting
13. Appendix — deployment cheat-sheet

1 What you will achieve

By the end of this guide you will have taken the **deployable network produced by the Training Guide** and run it as a continuous **advisory invalid-traffic scorer** that names the fraud family, shows its evidence, and recommends a candidate action for every ad event — but never blocks a publisher, withholds a payout, or accuses a person.

Safety ceiling for the whole guide

Everything below operates in ADVISORY / SHADOW mode. The scorer emits probabilities, evidence, and candidate actions; consequential actions (billing rejection, payout withholding, account blocking) stay disabled until first-party labelled validation, legal review, and an appeal/rollback process are in place.

2 Prerequisites — the trained bundle and runtime

- **A trained network** from the Training Guide — the model bundle directory (`model-descriptor.json`, `fallback-model.json`, the eight layer files, `thresholds.json`, `calibration.json`, `checksums.sha256`).
- **Java 17+** and the built worker (`mvn clean install`).
- **Your event feed** — a Kafka-style stream or an HTTP integration from your ad server, SSP/DSP, or mobile measurement partner.
- **An output destination** — a JSONL/audit sink for shadow mode, or your SIEM / data warehouse / analyst queue for operator-visible advisories.

Training first

If you do not yet have a trained network, run the companion Training Guide. Deployment consumes its output directory directly; the runtime bundle verifies its checksums on load.

3 The runtime scorer and the model bundle

Two classes do the work at runtime:

- `AdFraudModelBundle` loads the model bundle from the classpath (`model/advertising-fraud`), verifies `checksums.sha256`, and parses the calibrated heads, the non-linear hidden layer, the per-label thresholds, and the Platt calibration. If the bundle is missing or fails verification it falls back to a transparent rules-only model, so the service degrades safely rather than crashing.
- `AdFraudRuntimeScorer` extracts the typed features from each event (integrity, behavioural, supply-chain, quality, and the behavioural-evidence features), evaluates the hidden layer and the calibrated heads, and produces an `AdFraudDecision`.

The decision carries per-label probabilities, an overall invalid-traffic probability, the contributing reasons, a duplicate-event flag, the model version, and the recommended candidate action — everything an analyst needs to act and audit.

4 The deployable package (what training produced)

The trainer emits the **actual JNeopallium network**, not an abstract model. The bundle the runtime loads is:

Artifact	Role at deploy time
model-descriptor.json	Whole-network map: 8 layers, 22 neurons, frequency map, runtime classes
fallback-model.json	Calibrated heads + the non-linear hidden layer (featureNames, hidden, heads)
thresholds.json / calibration.json	Cost-aware per-label thresholds and Platt calibration
layer-0.json ... result-layer.json	The eight deployable layer files
checksums.sha256	Integrity manifest verified on load

Total network: **8 layers, 22 real neurons, 21 features** (8 base risks + 5 behavioural-evidence + 8 non-linear interaction units). Every neuron names a concrete class from the worker's ad-fraud package, so deployment loads this directory — it does not re-implement anything.

5 Option A — the HTTP scoring service

The simplest production shape is the bundled HTTP scoring service, which loads the model bundle and exposes a scoring endpoint plus Prometheus-style metrics. Point it at the verified bundle and POST canonical events:

```
# score a single canonical event
POST http://host:port/v1/ad-fraud/score
Content-Type: application/json
{ "eventId": "...", "eventType": "CLICK", "eventTime": 1800000000000, ... }

# operational metrics
GET http://host:port/metrics      # events_total, fraud_probability_histogram, ...
```

The response is the advisory decision (next section). For shadow mode, run the service read-only beside production and write its decisions to an audit sink; nothing in the response path touches your serving or billing.

6 Option B — the embedded worker network

To run the full eight-layer network inside the Jneopallium worker (for cluster-scale or to compose with other modules), launch the worker entry point with the four standard arguments and point `configuration.input.layermeta` at the generated bundle directory:

```
java -cp worker/target/worker-jar-with-dependencies.jar \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local \
  file:///opt/jneopallium/ad-fraud-model.jar \
  com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
  /opt/jneopallium/ad-fraud-context.json
```

Entry deserialises the context, loads the model JAR into an isolated class loader, and maps `local` / `http` / `grpc` to the single-process, HTTP-cluster, or gRPC application. The ad-fraud neuron, processor, and signal classes already ship in the worker, so the deployment JAR provides only your event source (`IInitInput`) and output sink (`IOutputAggregator`).

7 The advisory decision contract

Every scored event returns an auditable decision. Forward it to your SIEM, data warehouse, analyst queue, or billing-review system:

```
{
  "eventId": "clk-123",
  "modelVersion": "1.0.0-reference-advisory",
  "mode": "ADVISORY",
  "probabilities": {
    "clickInjection": 0.94, "attributionHijack": 0.91, "bot": 0.03, ...
  },
  "overallInvalidTrafficProbability": 0.95,
  "recommendedAction": "HOLD_PAYOUT_CANDIDATE",
  "reasons": [ "conversion observed before click", "click injection timing anomaly" ],
  "duplicateEvent": false,
  "evidenceCompleteness": 0.8
}
```

Candidate actions map from the response bands: ALLOW / LOG (low), REVIEW (0.30–0.60), RATE_LIMIT_CANDIDATE (0.60–0.75), HOLD_PAYOUT_CANDIDATE (0.75–0.90), ESCALATE_TO_ANALYST (0.90–1.0). Every action is a **candidate**; none is executed by the system.

8 Configure event sources and streaming input

Feed the scorer your live events. Two patterns:

- **HTTP integration** — your ad server / SSP / MMP POSTs each canonical event to `/v1/ad-fraud/score` and stores the returned decision.
- **Stream integration** — a site-specific `IInitInput` consumes your Kafka topics (bids, impressions, clicks, postbacks, retention/refund) and converts each record into the typed ad-fraud signals, preserving event time so delayed and out-of-order events still correlate.

Keep delayed-quality events (retention, refund, chargeback, uninstall) flowing — they are the slow truth that confirms or revises an earlier advisory, and they run on the slow loop by design.

9 Promote: shadow → advisory → bounded enforcement

Promote gradually. Each step earns the next; consequential actions stay disabled until the whole sequence has passed.

1. **Offline replay.** Score an exported event log without touching production. Deliver named invalid-traffic findings, an evidence timeline, an estimated wasted-spend impact, and a false-positive review.
2. **Shadow.** Read-only scoring beside production, decisions to an audit sink only. Compare advisories against first-party outcomes (chargebacks, MMP rulings) and analyst review.
3. **Advisory.** Surface decisions to analysts and billing review, once precision at your review capacity and calibration meet target.
4. **Bounded enforcement.** Only after first-party labelled validation, legal review, and an appeal/rollback process — and even then, narrowly scoped (e.g. hold a specific payout for review), never silent blocking or accusation.

10 Monitoring in production

- Events scored per minute; dropped, delayed, and out-of-order events.
- Per-label advisory rates; the fraud-probability histogram; false-positive review rate.
- Confirmed true-positive rate against first-party outcomes (chargebacks, MMP rulings).
- Calibration drift and model-drift signals (the slow monitoring cadence).
- Model version and bundle checksum used for every decision.

Every decision preserves its lineage: event id, per-label probabilities, contributing reasons, the model version, and the recommended candidate action.

11 Safety, privacy and rollback

- **Advisory by construction.** The response gate emits candidates only; automatic billing rejection, payout withholding, account blocking, and accusations are structurally disabled.
- **Privacy preserved.** Identifiers are pseudonymised (HMAC) before training and logs; raw IPs, precise geolocation, and user-agent strings are never exported; missing values are distinct from zero risk.
- **Safe degradation.** If the bundle is missing or fails checksum verification, the runtime falls back to a transparent rules-only model rather than failing the request path.
- **Rollback.** Keep the previous bundle available; roll back if false positives exceed budget, calibration drifts, or first-party outcomes diverge from advisories.

Honest limitation

The checked-in reference network is trained on a deterministic simulator plus weak public-source metadata. It is excellent for repeatable pipeline evidence and label-separation, but real-world accuracy requires first-party labelled production traffic, forward-time and unseen-publisher validation, and a documented review of data handling. It is advisory until then.

12 Troubleshooting

Symptom	Likely cause and fix
Service logs "rules-only fallback"	Bundle missing or checksum mismatch — re-export and redeploy the bundle
All probabilities look flat / wrong	Feature names changed; redeploy the matching bundle so the runtime augments correctly
Decisions emit a blocking action	Verify mode is ADVISORY; consequential actions must stay candidate-only
Delayed-quality labels never improve	Ensure retention/refund/chargeback events are fed on the slow loop
High false positives on a publisher	Tune per-label thresholds / review capacity; check for a baseline shift
Scores diverge from training	You deployed a stale bundle; redeploy the latest trained package

13 Appendix — deployment cheat-sheet

Score an event (HTTP service)

```
curl -s -X POST http://host:port/v1/ad-fraud/score \
-H 'Content-Type: application/json' \
-d '{"eventId":"clk-1","eventType":"CLICK","eventTime":1800000000000}'
curl -s http://host:port/metrics
```

Embedded worker network

```
java -cp worker/target/worker-jar-with-dependencies.jar \
com.rakovpublic.jneuropallium.worker.application.Entry \
local file:///opt/jneopallium/ad-fraud-model.jar \
com.rakovpublic.jneuropallium.worker.demo.fullrun.runtime.DemoJsonContext \
/opt/jneopallium/ad-fraud-context.json
```

Jneopallium Ad-Fraud Guardian · Deployment Guide. Consumes the network produced by the Training Guide. Safety mode: ADVISORY / SHADOW. Consequential actions require first-party validation, legal review, and an appeal/rollback process. License: BSD 3-Clause.